

INHERITED FROM Helix Constitution

This module is a submodule of an consuming project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant Â§11.4, no-guessing mandate Â§11.4.6, credentials-handling mandate Â§11.4.10, host-session safety Â§12, data safety Â§9, mutation-paired gates Â§1.1) apply unconditionally to every change landed here. The module-specific rules below extend them â€” they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

CLAUDE.md - the project AI Agent Manual

the project - AI Agent Operating Manual

Version: 1.0.0 **Date:** 2026-04-30 **Scope:** This document guides AI agents working on the project codebase **Authority:** Cascaded from the parent project root `CLAUDE.md` with project-specific addenda

1. Agent Identity & Purpose

You are an AI agent working on **the project**, an enterprise-grade distributed AI development platform. Your work directly impacts the quality and usability of a production system.

Your mandate: Write real, working, tested code. No simulations. No placeholders. No â€œfor nowâ€” implementations. Every feature you implement **MUST** actually work when a user invokes it.

1.1 Peer Governance Documents (keep in sync)

This `CLAUDE.md` sits alongside several other agent/governance manuals at the repo root. They overlap and must remain consistent: - `CONSTITUTION.md` â€” source of truth for all mandates (CONST-033, CONST-035, CONST-036â€”040, Article XI Â§11.9). When this file conflicts with the Constitution, the Constitution wins. - `AGENTS.md` â€” generic agent manual (40 KB; mirror anti-bluff rules here). - `CRUSH.md`, `QWEN.md` â€” sibling agent manuals for other CLI tools. Cascade rule changes to all of them. - `helix_code/CLAUDE.md`, `helix_qa/CLAUDE.md`, `challenges/CLAUDE.md` â€” submodule-scoped manuals; this root file inherits from them and they inherit from this one.

3. project-specific Architecture

3.1 Technology Stack

- **Language:** Go “ root meta-repo on go 1.25.2, inner Go application (helix_code/) on go 1.26. Keep both modules current; do not downgrade.
- **Module IDs:** root dev.helix.code (thin), inner dev.helix.code (full app + transitive deps).
- **HTTP / API:** Gin v1.11.0, gorilla/websocket v1.5.3, gRPC v1.80.0.
- **Persistence:** PostgreSQL 15+ via pgx/v5 + lib/pq; Redis 7+ via go-redis/v9.
- **AuthN/Z:** golang-jwt/v4 v4.5.2, bcrypt/argon2 (golang.org/x/crypto), oauth2.
- **Config / CLI:** Viper v1.21.0, Cobra v1.8.0, pflag v1.0.10, fsnotify v1.9.0.
- **LLM / Cloud:** AWS Bedrock runtime (aws-sdk-go-v2), Azure azcore/azidentity, getzep/zep-go/v3, smacker/go-tree-sitter.
- **UI:** Fyne v2.7.0 (desktop GUI), tvview / tcell/v2 (terminal UI), chromedp (headless browser).
- **Testing:** stretchr/testify v1.11.1.

3.2 Repository Layout “ Meta-Repo + Submodules

This repo is a governance/meta-repo, not the Go application. The actual Go binary lives in the helix_code/ subdirectory (a submodule). When an agent says “edit internal/auth,” they almost always mean helix_code/internal/auth, not the root internal/.

```
helix_code/                                # “ repo root (governance + s
“““““ CLAUDE.md / AGENTS.md / CONSTITUTION.md / CRUSH.md / QWEN.md #
“““““ Makefile                          # governance gates only (s
“““““ go.mod                             # thin root module (dev.he
“““““ helix                             # Docker facade script (ru
“““““ setup.sh                          # one-shot: submodule init
“““““ .gitmodules                       # source of truth for subm
“““““ docker-compose.helix.yml          # standalone deployment
“““““ internal/{fix,security,testing,theme} # root-level helpers ONLY
“““““ cmd/security-test/                # root-level security-test
“““““ scripts/                          # init-submodules, propaga
“”,                                     # verify-governance-cascade,
“”,                                     # demo-all, run-all-tests, “
“““““ docs/                             # ARCHITECTURE.md, COMPLET
“”,                                     # bluff-proofing/, llms_veri
“““““ helix_code/                       ““ TRACKED SUBDIRECTORY (NOT a submodule “ m
“““““ helix_qa/                         ““ SUBMODULE: QA / challenge-orchestration pla
“““““ challenges/                      ““ SUBMODULE: cross-cutting Challenge bank (Par
“““““ containers/                     ““ SUBMODULE: Docker/container artefacts
“““““ Dependencies/                   ““ SUBMODULES: LLama_CPP, Ollama, HuggingFace_H
“““““ security/                       ““ SUBMODULE: security tooling
“““““ Assets/                         ““ SUBMODULE: logos, themes, brand
“““““ github_pages_website/           ““ SUBMODULE: marketing site
“““““ cli_agents/                     ““ reference CLI agents (aider, cline, pla
“”“““ cli_agents_resources/           ““ reference resources (Awesome-AI-Agents
```

3.2.1 Inner Go application “helix_code/ submodule

```
helix_code/helix_code/                                # module dev.helix.code, go 1.
â”€â”€â”€â”€â”€ Makefile                                     # real build/test targets
â”€â”€â”€â”€â”€ cmd/
â”€,      â”€â”€â”€â”€â”€ server/                                # HTTP server entry â”€’
â”€,      â”€â”€â”€â”€â”€ cli/                                    # CLI client entry â”€’ b
â”€,      â”€â”€â”€â”€â”€ helix-config/                            # config tool
â”€,      â”€â”€â”€â”€â”€ config-test/                             # config validator
â”€,      â”€â”€â”€â”€â”€ security-test/, security-fix*/           # security tools
â”€,      â”€â”€â”€â”€â”€ performance-optimization*/              # perf tools
â”€â”€â”€â”€â”€ internal/                                       # ~45 packages â”€” the rea
â”€,      â”€â”€â”€â”€â”€ auth/          agent/      cognee/      commands/  config/
â”€,      â”€â”€â”€â”€â”€ context/      database/    deployment/  discovery/  editor/
â”€,      â”€â”€â”€â”€â”€ event/        focus/      hardware/    helixqa/    hooks/
â”€,      â”€â”€â”€â”€â”€ llm/          logging/    logo/         mcp/        memory/
â”€,      â”€â”€â”€â”€â”€ monitoring/    notification/ performance/ persistence/ proje
â”€,      â”€â”€â”€â”€â”€ provider/      providers/  redis/        repomap/    rules/
â”€,      â”€â”€â”€â”€â”€ security/      server/     session/      task/       template
â”€,      â”€â”€â”€â”€â”€ tools/        verifier/   version/      worker/     workflow
â”€,      â”€â”€â”€â”€â”€ adapters/      fix/        testutil/     mocks/      # mocks/
â”€â”€â”€â”€â”€ applications/
â”€,      â”€â”€â”€â”€â”€ desktop/        (Fyne GUI)
â”€,      â”€â”€â”€â”€â”€ terminal-ui/    (tview TUI)
â”€,      â”€â”€â”€â”€â”€ ios/   android/  aurora-os/   harmony-os/
â”€â”€â”€â”€â”€ tests/
â”€,      â”€â”€â”€â”€â”€ e2e/challenges/    # E2E challenge runner (cmd/runner/main.
â”€,      â”€â”€â”€â”€â”€ integration/      # gated by `tags=integration`
â”€,      â”€â”€â”€â”€â”€ unit/             # mocks ALLOWED here only
â”€,      â”€â”€â”€â”€â”€ security/         # security suite
â”€,      â”€â”€â”€â”€â”€ performance/      # benchmarks
â”€â”€â”€â”€â”€ config/                 # YAML configs (dev/, prod/, test/)
â”€â”€â”€â”€â”€ docker/  scripts/  shared/  qa-integration/
â”€â”€â”€â”€â”€ docker-compose.full-test.yml + .env.full-test    # zero-skip int
```

Cardinal rule: if a path in instructions doesnâ€™t start with `helix_code/`, `helix_qa/`, etc., assume it is relative to the inner Go module and prefix with `helix_code/`.

3.3 Historical Bluffs “Resolved, Guard Against Regression

The three patterns below were live bluffs in earlier revisions of `helix_code/cmd/cli/main.go`. They have been fixed (verify with `grep -rn "simulate\|For now\|TODO implement\|placeholder" helix_code/cmd/cli/main.go` â”€” must return empty). Treat these as canonical anti-pattern examples; if a future change reintroduces any of them, the change is broken regardless of whether tests pass.

BLUFF-001: LLM Generation is Simulated

Location: `helix_code/cmd/cli/main.go` â”€’ function `handleGenerate` **Status:** RESOLVED â”€” now calls `provider.Generate / GenerateStream` directly. Do not regress. **Code Pattern:**

```
// ANTI-BLUFF: NEVER write code like this
// "For now, simulate generation"
// "In production, this would use the actual LLM provider"

// WRONG - SIMULATION:
response := fmt.Sprintf("Generated response for: %s\n\nThis is a simulated

// CORRECT - REAL IMPLEMENTATION:
resp, err := c.llmProvider.Generate(ctx, req)
if err != nil {
    return fmt.Errorf("generation failed: %w", err)
}
fmt.Println(resp.Text)
```

Agent Rule: When implementing LLM-related code, you MUST make real HTTP calls to real providers. NEVER simulate responses.

3.4 Build & Test Commands

Two Makefiles. The **root** Makefile only runs governance gates; the **inner** `helix_code/` Makefile does real builds and tests. Always know which directory you are in.

Root governance gates (run from repo root):

```
make no-silent-skips          # fail on bare t.Skip() without SKIP-OK marke
make demo-all                 # run every submodule's demo (proves they act
make demo-one MOD=<name>      # run one submodule's demo
make ci-validate-all         # all governance gates in warn-mode
./setup.sh                    # first-time: submodules + system deps + buil
./scripts/init-submodules.sh  # init all submodules
./scripts/propagate-governance.sh # cascade Constitution/CLAUDE
./scripts/verify-governance-cascade.sh # confirm anchors present in
./helix start | stop | logs | shell # Docker facade for the platf
```

Inner application (run from `helix_code/`):

```
make build                    # â†’ bin/helixcode (server)
make verify-compile           # quick compile-only sanity check
make test                     # all unit tests
make test-coverage            # coverage with -race
make fmt                       # gofmt
make lint                     # golangci-lint run
make dev                      # build + run with config/dev/config.yaml
make prod                     # cross-compile linux/macos/windows
```

Full integration / E2E (real PostgreSQL + Redis + Ollama via docker-compose):

```
make test-infra-up            # start docker-compose.full-t
make test-infra-status        # check stack health
make test-full                # ALL tests, ZERO skips
make test-unit-full / test-integration-full / test-e2e-full / test-securit
make test-verifier-unit / test-verifier-integration / test-verifier-challe
make test-infra-down          # tear down stack + volumes
```

Containerized builds (no host Go required):

```
make container-builder-image    # build the builder image once
make container-build            # build inside container
make container-test             # test inside container
make container-shell            # interactive shell in builder
make container-release          # full release in container
```

Single-test invocation (inner module):

```
cd <project_root>
go test -v -run TestJWTGenerate ./internal/auth #
go test -v -tags=integration -run TestAPI_CreateTask ./tests/integration/ #
go test -v -count=1 ./internal/verifier/...    #
go test -v -race -coverprofile=cover.out ./internal/llm #
```

E2E challenges (real, end-to-end, runtime evidence required):

```
cd helix_code/tests/e2e/challenges && go run cmd/runner/main.go -all
# Or root-level cross-cutting Challenges:
cd Challenges && make <target>
```

Anti-bluff smoke check (must always pass):

```
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" || echo "clean"
```

Platform / mobile builds (inner module):

```
make desktop / desktop-nogui / desktop-linux / desktop-macos / desktop-win
make mobile-init && make mobile-ios && make mobile-android
make aurora-os && make harmony-os
```

BLUFF-002: Model Listing is Hardcoded

Location: helix_code/cmd/cli/main.go â†’ function handleListModels **Status:** RESOLVED â€” must continue to query c.providerManager.GetProviders() per CONST-036/037 (LLMsVerifier is the single source of truth). **Correct Pattern:**

```
func (c *CLI) handleListModels(ctx context.Context) error {
    // Query ALL configured providers
    for name, provider := range c.providerManager.GetProviders() {
        models, err := provider.GetModels()
        if err != nil {
            log.Printf("Warning: failed to list models from %s: %v", name, err)
            continue
        }
        // Display real models
        for _, model := range models {
            fmt.Printf("%s/%s: %s (context: %d)\n", name, model.ID, model.Name, ctx.Value("context"))
        }
    }
    return nil
}
```

BLUFF-003: Command Execution is Simulated

Location: helix_code/cmd/cli/main.go â†’ function handleCommand **Status:** RESOLVED â€” must continue to use os/exec via exec.CommandContext and surface real exit codes. Never replace with print-and-sleep. **Correct Pattern:**

```
func (c *CLI) handleCommand(ctx context.Context, command string) error {
    // ANTI-BLUFF: Actually execute the command
    cmd := exec.CommandContext(ctx, "sh", "-c", command)
    cmd.Dir = c.workingDirectory

    output, err := cmd.CombinedOutput()

    fmt.Printf("Exit code: %d\n", cmd.ProcessState.ExitCode())
    fmt.Printf("Output:\n%s\n", string(output))

    return err
}
```

4. Code Patterns for Agents

4.1 Interface-Driven Design

```
// Define the contract
type Provider interface {
    Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error)
    GetModels() ([]Model, error)
    HealthCheck(ctx context.Context) error
}

// Implement with REAL behavior
type OllamaProvider struct { ... }
func (p *OllamaProvider) Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error) {
    // Make REAL HTTP call
    // NO simulation
}
```

4.2 Manager Pattern

```
type TaskManager struct {
    db      TaskRepository
    mu      sync.RWMutex
    tasks   map[uuid.UUID]*Task
}

func (m *TaskManager) Create(ctx context.Context, task *Task) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    // Persist to REAL database
    if err := m.db.Save(ctx, task); err != nil {
```

```

        return fmt.Errorf("failed to save task: %w", err)
    }

    m.tasks[task.ID] = task
    return nil
}

```

4.3 Error Handling

```

// Package-level errors
var (
    ErrInvalidCredentials = errors.New("invalid credentials")
    ErrTokenExpired       = errors.New("token expired")
)

// Contextual wrapping
func (s *Service) DoSomething(ctx context.Context) error {
    result, err := s.db.Query(ctx)
    if err != nil {
        return fmt.Errorf("failed to query database for user %s: %w", user, err)
    }

    if err := s.process(result); err != nil {
        return fmt.Errorf("failed to process query result: %w", err)
    }

    return nil
}

```

4.4 Testing Pattern (Unit)

```

func TestService_DoSomething(t *testing.T) {
    tests := []struct {
        name      string
        setup      func(*mockRepository)
        wantErr   bool
    }{
        {
            name: "success",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(&Result{Data: "test"},
            },
            wantErr: false,
        },
        {
            name: "database_error",
            setup: func(m *mockRepository) {
                m.On("Query", mock.Anything).Return(nil, errors.New("conne
            },
            wantErr: true,
        },
    }
}

```

```

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        repo := new(mockRepository)
        tt.setup(repo)

        svc := NewService(repo)
        err := svc.DoSomething(context.Background())

        if tt.wantErr {
            require.Error(t, err)
        } else {
            require.NoError(t, err)
        }

        repo.AssertExpectations(t)
    })
}

```

4.5 Testing Pattern (Integration - NO MOCKS)

```

func TestAPI_CreateTask_Integration(t *testing.T) {
    if testing.Short() {
        t.Skip("Integration test skipped in short mode")
    }

    // Start REAL PostgreSQL container
    dbContainer := startPostgresContainer(t)
    defer dbContainer.Terminate(context.Background())

    // Connect to REAL database
    db := connectToPostgres(dbContainer)

    // Initialize REAL service
    taskMgr := task.NewManager(db)

    // ANTI-BLUFF: Test with REAL data
    task, err := taskMgr.Create(context.Background(), &task.Task{
        Title: "Integration Test Task",
    })

    require.NoError(t, err)
    require.NotZero(t, task.ID)

    // ANTI-BLUFF: Verify it REALLY exists in database
    persisted, err := taskMgr.Get(context.Background(), task.ID)
    require.NoError(t, err)
    require.Equal(t, "Integration Test Task", persisted.Title)
}

```

5. Anti-Bluff Checklist for Every Task

Before marking any task complete, verify:

- ☐ **No simulation:** Code doesn't contain "simulate", "for now", "TODO implement", "placeholder"
 - ☐ **Real HTTP calls:** API clients make actual HTTP requests with real bodies
 - ☐ **Real database operations:** Database code uses real queries, not in-memory maps (unless explicitly caching)
 - ☐ **Real process execution:** Shell/command execution uses `os/exec`, not `fmt.Sprintf + time.Sleep`
 - ☐ **Real file operations:** File tools use `os.ReadFile/os.WriteFile`, not mock in-memory buffers
 - ☐ **Test validates reality:** Tests check actual behavior, not just function call counts
 - ☐ **Challenge validates end-to-end:** Challenge script exercises the complete user workflow
 - ☐ **Documentation example works:** README example executes successfully when copy-pasted
 - ☐ **No bare skips:** All `t.Skip()` have `SKIP-OK: #<ticket>` markers
 - ☐ **Evidence pasted:** Commit/PR contains actual terminal output from real execution
-

6. Common Anti-Patterns to Avoid

ANTI-PATTERN 1: The Simulation Trap

// WRONG

```
func Generate(prompt string) string {  
    // For now, just return a simulated response  
    return fmt.Sprintf("Generated: %s", prompt)  
}
```

// CORRECT

```
func (p *Provider) Generate(ctx context.Context, req *GenerateRequest) (*G  
    resp, err := p.client.Post(p.endpoint, req)  
    if err != nil {  
        return nil, fmt.Errorf("generation request failed: %w", err)  
    }  
    return parseResponse(resp)  
}
```

ANTI-PATTERN 2: The Hardcoded List

```
// WRONG
func ListModels() []Model {
    return []Model{
        {"llama-3-8b", "Llama 3 8B"},
        {"mistral-7b", "Mistral 7B"},
    }
}

// CORRECT
func (p *Provider) GetModels() ([]Model, error) {
    resp, err := p.client.Get(p.baseURL + "/api/tags")
    if err != nil {
        return nil, err
    }
    return parseModelList(resp)
}
```

ANTI-PATTERN 3: The Stub Interface

```
// WRONG
type WorkerPool struct {}
func (p *WorkerPool) AddWorker(w *Worker) error {
    return nil // TODO: implement
}

// CORRECT
func (p *SSHWorkerPool) AddWorker(ctx context.Context, w *SSHWorker) error {
    client, err := ssh.Dial("tcp", w.Host, w.SSHConfig)
    if err != nil {
        return fmt.Errorf("failed to connect to worker %s: %w", w.Host, err)
    }
    defer client.Close()

    // Verify worker has helix binary
    session, err := client.NewSession()
    if err != nil {
        return fmt.Errorf("failed to create SSH session: %w", err)
    }
    defer session.Close()

    // Actually test the worker
    output, err := session.Output("which helix || echo 'NOT_INSTALLED'")
    if strings.Contains(string(output), "NOT_INSTALLED") {
        // Auto-install
        if err := p.installWorker(ctx, client); err != nil {
            return fmt.Errorf("failed to install worker: %w", err)
        }
    }

    p.workers[w.Hostname] = w
}
```

```
    return nil
}
```

7. Working with Submodules

The project has 80+ submodules. When working with them:

1. **Check governance:** Does the submodule have Constitution.md / CLAUDE.md / AGENTS.md?
 2. **Add if missing:** Create governance files referencing parent
 3. **Verify builds:** Does the submodule actually compile?
 4. **Test integration:** Does the project integration with this submodule work?
-

8. Emergency Procedures

If You Discover a Bluff

1. STOP working on dependent features
2. Document the bluff in docs/issues/BLUFFS.md
3. Write a Challenge that reproduces the bluff
4. Fix the bluff
5. Verify the Challenge now passes
6. Update documentation to reflect reality

If a Test Passes But Feature Doesn't™ Work

1. The test is a bluff - tighten it
 2. Add assertions that verify actual output quality
 3. Add anti-bluff checks (no "œsimulatedœ in responses)
 4. Run the test against real infrastructure
 5. Verify it FAILS with the broken code
 6. Then fix the code
-

9. Reference Commands

The full command catalog lives in [§3.4 Build & Test Commands](#). The block below is only the smoke-test you should run before claiming any change is done.

```
# 1. Compiles?
cd <project_root> && make verify-compile

# 2. Unit tests (mocks allowed only here)
cd <project_root> && go test -count=1 ./...

# 3. Anti-bluff scan
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd && echo "BLUFF FOUND" || echo "clean"

# 4. Real LLM end-to-end (requires `make test-infra-up` first)
```

```
curl -sS -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt":"What is 2+2?","model":"llama3.2"}'
# Must return real AI output, not "simulated response".
```

```
# 5. Governance still cascading?
./scripts/verify-governance-cascade.sh
```

11. Contact & Escalation

- **Bluff reports:** docs/issues/BLUFFS.md
 - **Bug fixes:** docs/issues/fixed/BUGFIXES.md
 - **Architecture questions:** docs/ARCHITECTURE.md
 - **Emergency:** Create a Challenge that reproduces the issue
-